Szyfr Vernama

Marcin Kocot

Zadanie wykonano w języku Kotlin, przy użyciu build-toola Bazel.

Generator liczb losowych

Użyto własnej implementacji Blum-Blum-Shuba

```
        fun next(): Int {
            var ret = 0
            repeat(Int.SIZE_BITS) {
                ret = ret shl 1
                if (nextBoolean()) ret += 1
            }
            return ret
        }

        fun nextBoolean(): Boolean =
            ((lastX * lastX) % n).let {
                this.lastX = it
                it % BigInteger.TWO == BigInteger.ZERO
            }
```

Wartości *p* i *q* wybierane są tak, żeby ich reszta z dzielenia przez 4 wynosiła 3

```
        fun getMod4Eq3ProbablePrime(bitLen: Int, rGen: Random): BigInteger =
            generateBigIntegersUntil(bitLen, rGen) {
                it % BigInteger("4") == BigInteger("3")
            }
```

Generacja klucza

Klucz składa się z kolejnych bajtów wygenerowanych opisanym powyżej Blum-Blum-Shubem

```
        object KeyGen {
            val RGen = BlumBlumShub()

            fun generateKey(byteLen: Int): BitByteList {
                val bytes = ByteArray(byteLen).apply {
                    RGen.nextBytes(this)
                }
                return BitByteList.from(bytes)
            }
        }
```

Testowanie klucza

Zaimplementowano dwa testy, oba zaproponowane przez NIST.

Monobit Test

Weryfikuje proporcje zer i jedynek

```
        fun monobitTest(key: BitByteList): Pair<Boolean, String> {
            val sN = key.toList().fold(0) { acc, bitByte ->
                acc + (2 * bitByte.bitSum() - 8)
            }
            val sObs = sN / sqrt(key.bitN.toDouble())
            val pVal = Erf.erfc(sObs / sqrt(2.0))
            return (pVal >= 0.01) to "p-value = $pVal"
        }
```

Serial Test

Weryfikuje powtarzalność podciągów

```
        fun serialTest(key: BitByteList): Pair<Boolean, String> {
            val n = key.bitN
            val m = 3
            val e = key.toBinString().let {
                "$it${it.take(m-1)}"
            }
            val fiSq0 = e.calculateFiSq(n, m)
            val fiSq1 = e.calculateFiSq(n, m - 1)
            val fiSq2 = e.calculateFiSq(n, m - 2)

            val delta = fiSq0 - fiSq1
            val deltaSq = fiSq0 - 2 * fiSq1 + fiSq2

            val pVal1 = IncompleteGamma.Upper.value((2.0).pow(m - 2), delta)
            val pVal2 = IncompleteGamma.Upper.value((2.0).pow(m - 3),
deltaSq)

            return (pVal1 >= 0.01 && pVal2 >= 0.01) to "p-values = ($pVal1,
$pVal2)"
        }
```

Działanie

Testowanie odbywało się poprzez zaszyfrowanie tekstu w języku Kantońskim.

```
1. 香港人口稠密，擁有眾多島嶼，加上道路使用率之高，位居世界前列，為了應付龐大的人口交通流量，
香港興建了多座跨海大橋
2. e9a699e6b8afe4babae58fa3e7a8a0e5af86efbc8ce69381e69c89e79cbee5a49...
3. f16b0309f7fb0750f63f07639478608e1c1c19be1d4870c7874001063fe0f255c...
4. 18cd9aef4f54e3ea4cda88c073d0c06bb39af60291aee34661dc88e1a35e17f15...
5. e9a699e6b8afe4babae58fa3e7a8a0e5af86efbc8ce69381e69c89e79cbee5a49...
6. 香港人口稠密，擁有眾多島嶼，加上道路使用率之高，位居世界前列，為了應付龐大的人口交通流量，
香港興建了多座跨海大橋
7. diff = 0
8. Is key random (monobit test)? Yes [p-value = 1.1727406534372888]
9. Is key random (serial test)? Yes [p-values = (0.05368004689518295,
0.110539655137410 08)]
```

Linijki przedstawiają kolejno:

1. Tekst otwarty
2. …i jego binarną formę
3. Klucz
4. Binarna forma tekstu zaszyfrowanego
5. Binarna forma tekstu odszyfrowanego (taka sama jak w punkcie **2**)
6. Tekst odszyfrowany
7. Różnica między tekstami (0 oznacza że są takie same)
8. Wynik testu Monobit
9. Wynik testu Serial